Чекмасов Егор
Вариант 90

$F(x) = x^2$

$E = [0, 4]$

$$F(x) = \begin{cases} 0, & x \le 1 \\ 1, & 1 < x \le 2 \\ 2, & x > 2 \end{cases}$$

Аналитический метод

1) 1) $[0, 4]$ – брус => область определения – измеримое мн-во

2) $f(x) = x^2$ – непрерывна на всей числовой прямой => непр. на E

Из п. 1 и 2 => $f(x)$ – измерима по Лебегу т.к. прообраз открытого множества открыт => измерим.

2)

Кол-во разбиений $= 2^n$

Длина разбиения $= \frac{4}{2^n}$

$x_{k-1} = \frac{4(k-1)}{2^n}$ – координата левой части на данном разбиении

$\Downarrow$

$x_{k-1} \le x \Rightarrow x_{k-1}^2 \le x^2 \Rightarrow f_n(x) \le f(x)$

Тогда:

Возьмём $f_n = \sum_{k=1}^{2^n} \left(\frac{4(k-1)}{2^n}\right)^2 \cdot \chi_{\left[\frac{4(k-1)}{2^n}, \frac{4k}{2^n}\right)}$

Очевидно: $0 \le f(x) - f_n(x) = x^2 - x_{k-1}^2 = (x - x_{k-1})(x + x_{k-1})$

- $x - x_{k-1} < \frac{4}{2^n}$
- $x + x_{k-1} \le 4 + 4 = 8$

$\Rightarrow 0 \le f(x) - f_n(x) < \frac{32}{2^n} \Rightarrow f_n(x) \xrightarrow[n \to \infty]{} f(x)$

3

- $x_{k-1}^2 \geq 0 \Rightarrow f_n(x) \geq 0$
- $f_n(x) \xrightarrow[n\to\infty]{} f(x)$ (из п.2)
- Если отрезки делить не на 2^n а на 2^{n+1} то каждый разделится пополам и на правой половине значение увеличится.

$\Downarrow$

$f_n \leq f_{n+1}$

По теореме Беппо Леви:

$$\lim_{n\to+\infty} \int_E f_n \, d\mu = \int_E f \, d\mu$$

$\Downarrow$

$$\lim_{n\to+\infty} \int_E f_n \, d\mu = \int_0^4 x^2 \, dx = \frac{64}{3} = 21\frac{1}{3}$$

4

$$F(x) = \begin{cases} 0, & x \leq 1 \\ 1, & 1 < x \leq 2 \\ 2, & x > 2 \end{cases}$$

- С ростом x - $F(x)$ - увеличивается или остается постоянным

$\Downarrow$

$F(x)$ - возрастающая

- Функция непрерывна на всей числовой прямой $\Rightarrow$ непр. слева.

$F(x)$ задаёт меру Лебега-Стильтьеса μ_F.

5

$$\int_E f \, d\mu_F$$

$$\left.\begin{array}{l} F(1) = 0 \\ F(1+0) = 1 \end{array}\right. \Rightarrow \mu_F(\{1\}) = 1$$

$f(1) = 1$

$$\left.\begin{array}{l} F(2) = 1 \\ F(2+0) = 2 \end{array}\right. \Rightarrow \mu_F(\{2\}) = 1$$

$f(2) = 4$

На остальной части $\mu F = 0$ т.к. $F(x)$ там постоянна.

Тогда:

$$\boxed{\int_E x^2 \, d\mu_F = 1 \cdot 1 + 1 \cdot 4 = 5}$$

Численный метод

Импорт библиотек и создание функций

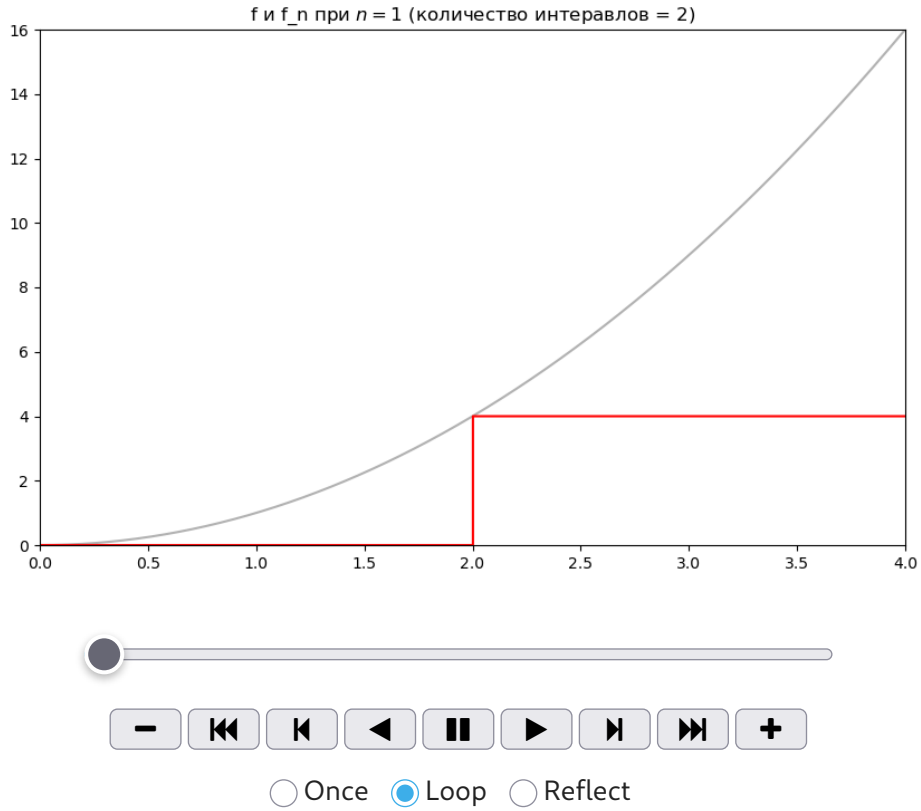
In [29]:
```
import numpy as np
import matplotlib.pyplot as plt

# Зададим исходную функцию, с которой будем сравнивать приближение
def f(x):
    return x**2

# Сделаем простую функцию f_n: на каждом отрезке берем левый конец
def f_n(x, n):
    # Разобьем [0; 4] на 2^n одинаковых интервалов
    num_intervals = 2**n

    # Отдельно обработаем число, потому что для него не работают векторны
    if np.isscalar(x):
        k = int(x * num_intervals) // 4
        # Ограничим индекс, чтобы не выйти за последний интервал
        k = max(0, min(k, num_intervals - 1))
        x_k_min_1 = 4 * k / num_intervals
    else:
        # Для массива сразу найдем шаг и номера интервалов
        dx = 4 / num_intervals
        k = np.clip(np.floor(x / dx).astype(int), 0, num_intervals - 1)
        x_k_min_1 = k * dx

    # Вернем значение f в левом конце интервала
    return x_k_min_1**2
# Запомним точное значение интеграла, чтобы потом сравнить
integral_real_1 = 64/3
integral_real_2 = 5
```

2.1 Изобразим график f и f_n (для сравнения) при n от 1 до 10

In [30]:
```
# Подключим инструменты для анимации и показа в ноутбуке
from matplotlib.animation import FuncAnimation
from IPython.display import HTML

# Настроим область графика перед первым кадром
def init():
    ax.set_xlim(0, 4)
    ax.set_ylim(0, 16)
    return step_plot,

# Для каждого n будут пересчитываться значения ступенчатой функции
def update(n):
    count = len(x_vals)
    y_step = [0] * count

    for i in range(count):
```

```
        x = x_vals[i]
        # В каждой точке возьмем значение f_n
        y_step[i] = f_n(x, n)
    step_plot.set_data(x_vals, y_step)
    ax.set_title(f'f и f_n при $n = {n}$ (количество интеравлов = {2**n})
    return step_plot,

# Подготовим окно и сетку точек на отрезок [0; 4]
fig, ax = plt.subplots(figsize=(10, 6))
x_vals = np.linspace(0, 4, 1000)

# Сначала нарисуем исходную f(x), потом добавим пустую линию для f_n
ax.plot(x_vals, f(x_vals), color='black', alpha=0.3, label='$f(x) = x^2$'
step_plot, = ax.step([], [], where='post', color='red', label='Простая фу

# Соберем анимацию для n от 1 до 10
ani = FuncAnimation(fig, update, frames=range(1, 11), init_func=init, bli

# Закроем статичную картинку и выведем именно анимацию
plt.close()
HTML(ani.to_jshtml())
```

Out[30]:



2.2

Вычислим интеграл Лебега для f_n при разных n и сравним с результатом, полученным в пункте 1.3

In [31]:

```
# Посчитаем интеграл Лебега для f_n по готовой формуле
def integral_l(n):
```

```
    intervals = 2**n
    return 64 / 3 - (32 / intervals) + (32 / (3 * intervals**2))

# Прогоним несколько n и сравним с точным интегралом
for n in [10, 100, 1000]:
    res = integral_l(n)
    # Посчитаем модуль отклонения от аналитического значения
    difference = abs(res - integral_real_1)
    print(f"n = {n:4}: Интеграл = {res:.8f} (Отклонение от аналитического
```

```
n =   10: Интеграл = 21.30209351 (Отклонение от аналитического результата:
3.12398275e-02)
n =  100: Интеграл = 21.33333333 (Отклонение от аналитического результата:
0.00000000e+00)
n = 1000: Интеграл = 21.33333333 (Отклонение от аналитического результата:
0.00000000e+00)
```

2.3

Вычислим интеграл Лебега-Стилтьеса для f_n при разных n и сравним с результатом, полученным в пункте 1.5

In [32]:
```
# Посчитаем интеграл Лебега-Стилтьеса через точки скачков
def integral_ls(n):
    # Зададим точки скачков и их веса
    jump_points = [1, 2]
    jump_weights = [1, 1]
    res = 0
    # Пройдемся по всем скачкам и суммируем вклад каждой точки
    for i in range(len(jump_points)):
        x = jump_points[i]
        w = jump_weights[i]
        # Берем значение функции в точке скачка
        value = f_n(x, n)

        res = res + value * w
    return res

# Проверим результат для больших n и сравним с точным значением
for n in [10000, 100000, 1000000]:
    res = integral_ls(n)
    # Посчитаем отклонение от аналитического результата
    difference = abs(res - integral_real_2)
    print(f"n = {n:4}: Интеграл = {res:.8f} (Отклонение от аналитического
```

```
n = 10000: Интеграл = 5.00000000 (Отклонение от аналитического результата:
0.00000000e+00)
n = 100000: Интеграл = 5.00000000 (Отклонение от аналитического результат
а: 0.00000000e+00)
n = 1000000: Интеграл = 5.00000000 (Отклонение от аналитического результат
а: 0.00000000e+00)
```